and passes control to loadFromData. Since Project#load returns the result of the then call, which is itself a promise, the test waits for that promise to resolve (project.load().then(() => {) so that you can verify that the results are as expected.

Verifying Test Doubles

Using test doubles is more than just setting stubbed behaviors to replace real behaviors; test doubles are also used to verify behavior by showing that various methods were called as side effects of the code under test. In RSpec, this verification is done with expect and have_received. In testdouble.js, it's done with the td.verify method.

There is a side effect in this code. When you move a task up or down, the JavaScript still makes an Ajax call back to the server to register the change server-side. You don't really care about the result of that call (at least not for the purposes of this test), but you do care that it gets made. So, you can put that validation into the tests by changing the "move up" and "move down" tests.

First some imports at the top of the file:

js_jasmine/03/spec/javascripts/project_spec.js

```
import {Project} from "../../app/javascript/packs/project.js"
import {Task} from "../../app/javascript/packs/task.js"
import {TaskUpdater} from "../../app/javascript/packs/task_updater.js"
import td from "testdouble/dist/testdouble"
import tdJasmine from "testdouble-jasmine"
tdJasmine.use(td)
```

Then the changed tests themselves (the two tests here use slightly different validation syntax, which you'll see more about in a moment):

js_jasmine/03/spec/javascripts/project_spec.js

```
it("can move a task up", () => {
  const FakeUpdater = td.constructor(TaskUpdater)
  project.tasks[1].updater = new FakeUpdater()
  project.tasks[1].moveUp()
  expect(project.firstTask().name).toEqual("Middle Project")
  expect(project.tasks[1].name).toEqual("Start Project")
  expect(project.lastTask().name).toEqual("End Project")
  td.verify(FakeUpdater.prototype.update("up"))
})
it("can move a task down", () => {
  const FakeUpdater = td.constructor(TaskUpdater)
  project.tasks[1].updater = new FakeUpdater()
  project.tasks[1].moveDown()
  expect(project.firstTask().name).toEqual("Start Project")
  expect(project.tasks[1].name).toEqual("End Project")
  expect(project.lastTask().name).toEqual("Middle Project")
  expect().toVerify(FakeUpdater.prototype.update("down"))
})
```